

클라우드기반 대출 상환 위험 모니터링 플랫폼 개발

팀명 : RISKFINDER

팀원 : 컴퓨터인공지능학부 신서현

컴퓨터공학부 박민호

컴퓨터공학부 박준영

컴퓨터공학부 임다미

목차

1 과제 개요

2 시스템 설계

3 주요 기능

4 팀원 소개 및 협업 과정

chapter 1

과제 개요

문제 인식

개발 목적

프로젝트 소개

대한민국, 'GDP 대비 가계부채 압도적 1위' 국가

대한민국 가계부채 비율은 세계 1위로, 국가 경제를 위협하는 최대 리스크 요인입니다.

한국 가계부채 비율, 새 기준 적용에도 여전히 세계 최고 수준

송고 2024-06-09 06:01



한지훈 기자
+ 구독



신호경 기자
+ 구독



민선희 기자
+ 구독

작년 말 93.5%... '100% 하회' 목표 달성에도 '하향 안정화' 과제
기업부채 비율은 4→5위... 일본과 자리바꿈

(서울=연합뉴스) 신호경 한지훈 민선희 기자 = 국민계정 통계의 기준 연도 개편으로 국민총생산(GDP) 등의 지표가 크게 개선됐으나 한국의 GDP 대비 가계부채 비율은 여전히 세계 주요국 가운데 1위 수준으로 나타났다.

출처 : 연합뉴스 (2024.06)

경제

가계 빚, 3.6년 만에 GDP 아래로...韓, 4년째 세계 최대 '가계부채국'

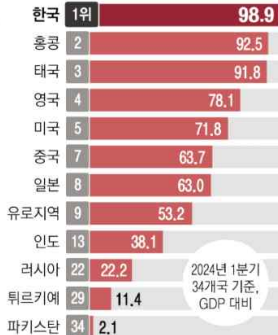


최재전, 신동아, 이재성 기자
입력 2024-05-10 01:09 | 수정 2024-05-10 01:09

가계부채 비율 98.9% 기록

선진국 평균보다 29%p 더 높아
"실질 금리 올라 빚 부담 안 줄어"
기업부채 비율은 123% 더 심각
"정부가 기업들 더 세심히 살펴야"

주요국 가계부채 비율 (단위: %)



(자료: 국제금융협회(IIF))

출처 : 서울신문 (2024.05.10)

69개월 만의 최고치, 이미 커져버린 '연체율 경고등'

고금리 장기화로 인해 은행 연체율이 5년 9개월 만에 최고치를 기록하며
상환 능력이 붕괴되고 있습니다. 이로 인해 매년 수조 원이 부실채권 정리 비용으로 사라
지며 부실 채권으로 인한 은행들은 자산 건전성 위기를 맞고 있습니다.

8월 은행 연체율 0.53%...69개월 만에 최고치

전월비 0.06%p ↑ "고금리-경기침체 여파로 부실 대출 증가"

박대현 기자 | pdy@newsprime.co.kr | 2024.10.18 09:45:11

원화대출 연체율 추이(13.1월~24.8월)



※ 원화대출 연체율 추이(13.1월~24.8월) ※ 금융감독원

[프라임경제] 계속되는 고금리와 경기침체에 국내은행의 원화대출 연체율이 69개월 만에 최고치를 달성했다.

18일 금융감독원(이하 금감원)이 발표한 '국내은행의 원화대출 연체율 현황'에 따르면, 8월 말 국내은행의 원화대출 연체율(1개월이상 원리금 연체기준)은 0.53%로 전월 말 대비 0.06%p(포인트) 상승했다.

출처 : 프라임경제 (2024.10.18)

경북의 바다

고금리의 그늘... 5대은행, 2024년 부실채권 7조 털었다

입력 2025-02-02 20:39:00 | 수정 2025-02-02 19:52:22

윤호 기자 yoh@nbsnews.com

못 갚는 대출 늘자 상매각 2년 새 3배로
전년 대비 연체율·NPL 여전히 상승세
은행권, 경기 불안 겹쳐 건전성 관리 고삐

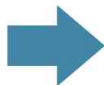
주요 시중은행이 지난해 7조원이 넘는 부실채권을 상각 또는 매각해 털어낸 것으로 나타났다. 고금리·고물가 영향으로 은행 빚을 제때 갚지 못한 가계와 기업이 증가하면서 은행권의 부실채권 규모는 2년 사이 3배로 뛰었다.

2일 금융권에 따르면 5대 은행(KB·신한·하나·우리·NH농협)은 지난해 7조1019억원 상당의 부실채권을 상각 또는 매각했다. 2023년 5조4544억원의 약 1.3배, 2022년 2조3013억원의 약 3배 규모다.

출처 : 조선비즈 (2025.10.16)

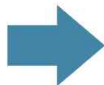
인적 자원의 한계에 따른 리스크 관리 고도화 필요

심사·모니터링 업무량 증가



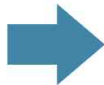
인력 부담 확대

반복적·정형화된 업무



자동화 수요 증가

데이터 종류·규모 확대



분석 난이도 상승

기존 인력 중심 방식만으로는 '선제적 리스크 대응'에 한계

즉, 은행의 자산 건전성을 유지하고
부실 위험을 최소화하기 위한 서비스의 도입이 필요함



사후 관리를 넘어, AI 기반의 '선제적 위험 예측'

" AI 기반 대출 리스크 관리 서비스 개발 "

1

AI 기반 사전 예측

연체가 발생하기 전에 고위험
고객을 자동으로 찾아내고
원인을 도출합니다.

2

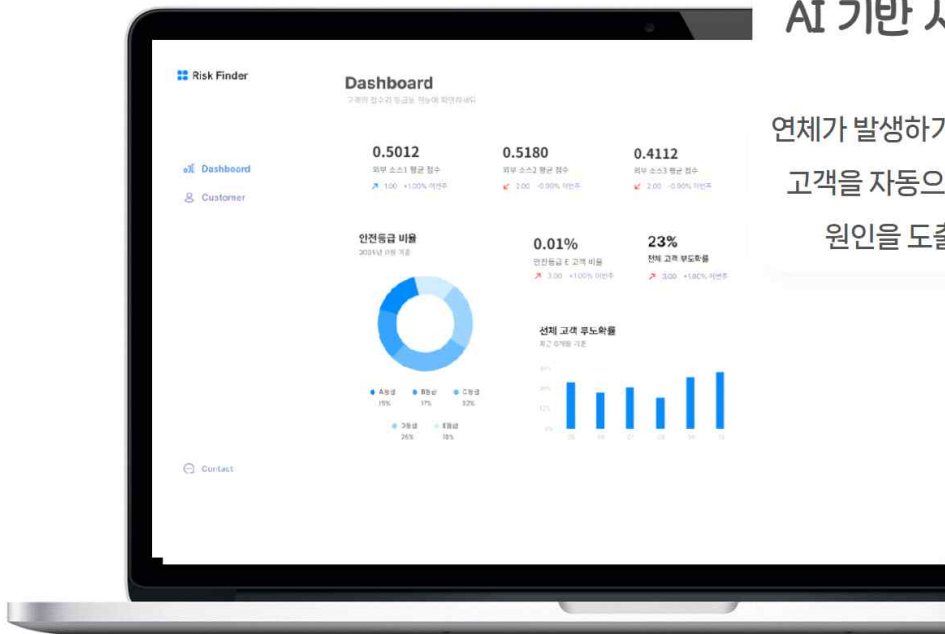
클라우드 기반 자동 모니터링

정해진 주기에 맞춰
AI 예측 모델을
자동으로 실행합니다.

3

관리자 맞춤형 시각화

복잡한 위험 데이터를
대시보드와 그래프로 한눈에
파악합니다.



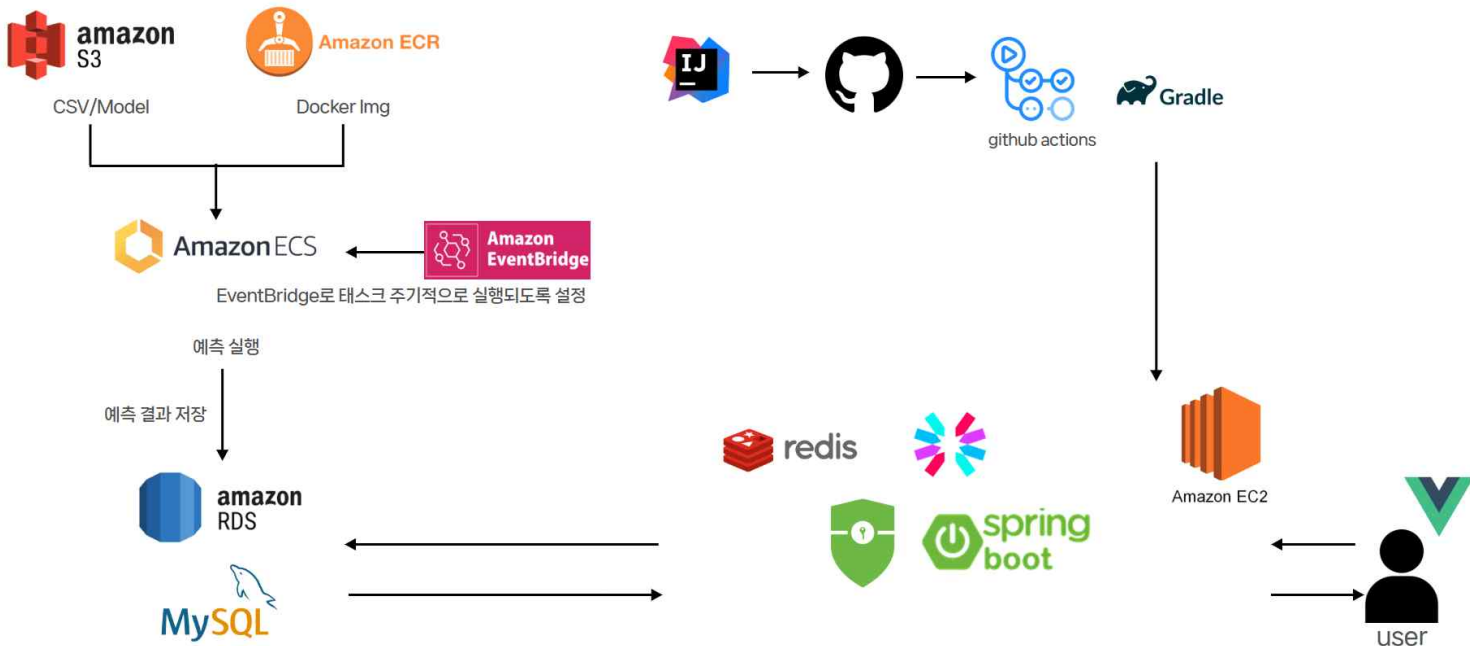
chapter 2

시스템 설계

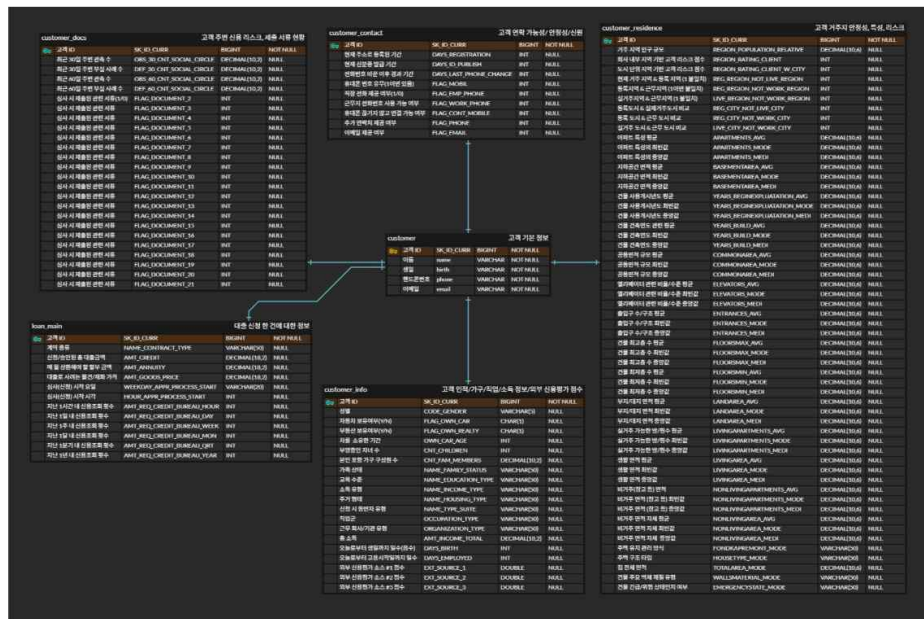
시스템 아키텍처

ERD

2. 시스템 설계



customer 테이블의 고객 고유 식별자인 SK_ID_CURR를 통해 5개의 상세 정보 테이블' 연결



고객 기본 정보(customer)

고객의 핵심 식별 데이터

금융/신용 관련 정보 테이블

대출 및 신용 이력
(loan_main)

신상 및 고객 신원 안정성
(customer_info, contact)

자산 및 잠재적 리스크 지표
(customer_residence, docs)

Service ERD



예측 결과 테이블

최신 상태 관리
(scoring_results)

이력 추적
(score_history)

운영 및 대응 테이블

알림
(alerts)

관리자
(users)

chapter 3

주요 기능

개발 방법

페이지별 기능 설명

AI 예측 파이프라인 개요

데이터 고도화 -> 모델 최적화 -> 설명 가능한 서비스

1. Data & Feature

- 1
고객의 대출 관련 정보가 담긴
7개의 데이터 레이블에서 데이터 수집
- ↓
- 2
피처 엔지니어링을 통한
200+ 개의 변수 생성

2. Modeling

- 3
Optuna를 사용한
하이퍼파라미터 최적화
- ↓
- 4
XGBoost, LightGBM, CatBoost
3개의 모델을 학습
- ↓
- 5
3개의 모델을 조합
(앙상블)

3. Service

- 6
점수 내기에 필요한 시스템들을
하나의 파이프라인으로 통합
- ↓
- 7
Batch Scoring을 통한
고객의 점수 계산
- ↓
- 8
위험 등급과
주요 위험 원인 출력

DATA 수집 및 전략

출처 : Home Credit Group

- 약 30만 명의 실제 대출 고객 정보를 익명화 하여 공개한 데이터
- 정적 정보(신상)와 동적 정보(상환 이력)가 결합된 7개의 csv파일 사용

핵심데이터

application_train/test.csv : 고객 기본 정보 (소득, 직업, 가족 구성, 주거 형태 등)

내부 금융 이력

previous_application.csv : 해당 금융사에서의 과거 대출 신청 이력 (승인/거절 내역)

installments_payments.csv : 대출 상환 내역(예정일, 실제 납부일, 예정 금액, 실제 납부 금액)

credit_card_balance.csv : 월별 신용카드 잔액, 사용 한도, 인출 내역

POS_CASH_balance.csv : 월별 현금 대출 및 할부(POS) 잔액 정보

외부 신용 정보

bureau.csv : 타 금융기관(은행, 카드사)에서의 신용 거래 요약 정보

bureau_balance.csv : 타 금융기관 신용 거래의 월별 잔액 및 연체 상태

피처 엔지니어링

데이터 고도화 및 파생 변수 생성

자금난이 시작되는 행동패턴을 모델에 학습시키기 위함

기본 피처 엔지니어링

- 고객 기본 정보 데이터 전처리
- 신용 정보원 이력 데이터 병합
- 과거 대출 신청 데이터 분석

고급 피처 엔지니어링

- 할부 납부 이력 분석
- 신용카드 이용 패턴 분석
- 금융 리스크 지표 계산
- 신용 유형별 연체율 집계

Target Encoding

- 직업 등 범주형 데이터를 수치화
- 5-Fold CV를 적용하여 Data Leakage(과적합) 방지

예시)

자금 여력 : 원본 데이터 [AMT_INSTALLMENT - AMT_PAYMENT] => 파생 변수 [PAYMENT_DIFF]

납부 예정액

실제 납부액

납부 부족액

앙상블 기반 모델링 및 최적화

하이퍼 파라미터 최적화

- Optuna 프레임워크 활용
- 각 모델당 50회 반복 실험으로 최적값 자동 탐색



3-Model 앙상블

$$\text{Score} = 0.35X + 0.35L + 0.30C$$

XGBoost (35%): 일반화 성능 (안정성)

LightGBM (35%): 대용량 학습 속도

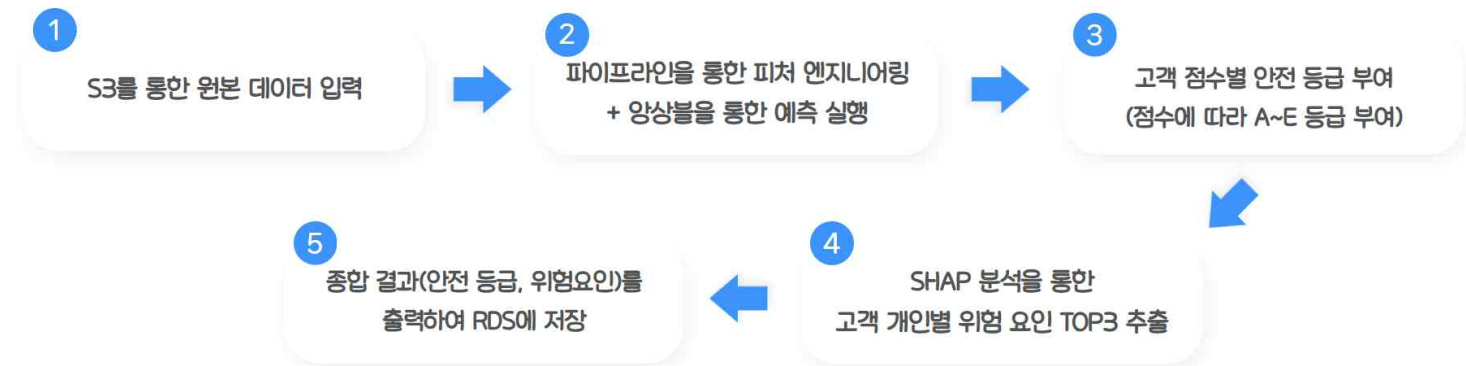
CatBoost (30%): 범주형 데이터 강점

단일 모델의 편향을 줄이고
정형 데이터에서 가장 안정적이고 높은 성능을 내기 위해
최적의 비율로 세 가지 모델을 결합함

하나의 파이프라인으로 통합 -> 배치 스코어링 시스템에 탑재

모델 결과 후처리 및 등급 체계

자동화된 배치 스코어링 및 결과 해석



	SK_ID_CURR	score	grade	top3_features
▶	100001	0.0625	A (안전)	◊ EXT_SOURCE_MEAN △ bureau_DAYS_CREDIT_max △ inst_AMT_PAYMENT_sum
	100005	0.1314	B (양호)	△ inst_AMT_PAYMENT_sum △ bureau_DAYS_CREDIT_max △ EXT_SOURCE_MEAN
	100013	0.0150	A (안전)	◊ EXT_SOURCE_MEAN ◊ ANNUITY_TO_CREDIT_RATIO ◊ inst_AMT_PAYMENT...
	100028	0.0366	A (안전)	◊ EXT_SOURCE_MEAN △ bureau_DAYS_CREDIT_max ◊ CREDIT_TO_GOODS_...
	100038	0.1071	B (양호)	△ EXT_SOURCE_MEAN ◊ CREDIT_TO_GOODS_RATIO △ bureau_DAYS_CREDI...

"설명가능성(SHAP)"

안전

고객 A

ID : 100057

나이 : 46세

성별 : 남성

교육 : 대졸

직업 : IT 전문직

근속 : 7.1년

자산 : 차량(1대)

부동산(1채)

VS

위험

고객 B

ID : 103420

나이 : 28세

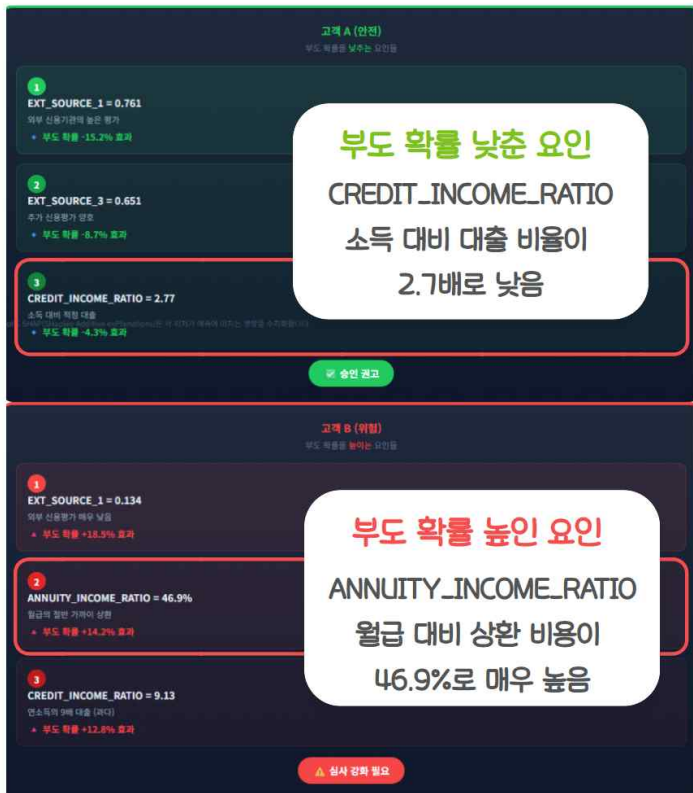
성별 : 여성

교육 : 고졸

직업 : 의료 보조

근속 : 6.8년

자산 : 부동산(1채)



Infra / DevOps

서비스 아키텍처

하이브리드 배포 전략을 통한 고효율 웹 서비스 및 AI 파이프라인 구축

1. Web Zone



main에 push -> github actions -> EC2(Frontend/Backend)

2. AI Zone



Local -> Docker Build -> ECR push -> S3(CSV/MODEL) + ECR -> ECS(EventBridge) -> RDS

1

하이브리드 배포

1. Web : github actions 기반 자동배포(ci/cd)로 개발 생산성 up
2. AI : 모델 검증 후 ECR로 직접 업로드 해서 운영 안정성 확보

2

부하 Isolation

무거운 AI 연산을 ECS로 격리하여, 대출 심사 중에도 웹 속도 저하 방지

3

DATA 파이프라인

S3의 대용량 CSV/PKL을 EventBridge 주기에 맞춰 분석하고
결과만 RDS에 적재

Infra / DevOps

엔지니어링 전략

비용 최적화

Architecture



Event Bridge



ECS Fargate

why ?

고성능 AI 서버를 24시간 켜두는 것은 낭비

데이터가 처리되는 시점에만 컨테이너를 실행하는 Serverless(On-Demand) 구조로 비용 절감

엔지니어링 전략

부하 격리

Architecture



Web (EC2)



AI (ECS)

why ?

AI 모델이 CPU를 점유하면 웹 서버가 멈출 위험이 있음

물리적 서버 분리를 통해 대용량 데이터 처리 중에도 웹 서비스의 응답 속도 보장

엔지니어링 전략

운영 이원화

Architecture

Web => CI/CD로 자동 배포

AI => 로컬에서 수동으로 ECR에 직접 업로드

why ?

작은 배포가 필요한 웹

VS

검증이 필요한 금융 AI 모델

웹은 CI/CD로 속도를,
AI는 수동 검증으로 안정성을 선택한
하이브리드 운영

The image displays four logos arranged in a 2x2 grid. The top-left logo is the Java logo, featuring a stylized coffee cup with steam and the word 'Java' in orange. The top-right logo is the Spring Boot logo, consisting of a green power button icon and the text 'spring boot' in green. The bottom-left logo is the MySQL logo, showing a blue mouse cursor icon and the text 'MySQL' in blue and orange. The bottom-right logo is a colorful starburst or flower-like shape made of eight triangular segments in various colors (red, orange, yellow, green, blue, purple, pink, and cyan).

핵심기능

1. 인증 시스템

- 1) 이메일 중복 검사: 가입 전 등록된 이메일 여부 확인
- 2) SMTP 메일 발송: JavaMailSender를 활용한 인증 코드 전송
- 3) Redis 캐싱 전략
 - 인증 코드를 Redis에 저장 (TTL 설정으로 보안 강화)
 - verifyCode 성공 시 "인증됨(Verified)" 상태를 Redis에 기록하여 가입
- 4) 보안 가입: 비밀번호 암호화(BCrypt) 및 DTO Validation(@Pattern, @N

2. 표준화된 에러 처리

- ♥ Global Exception Handling
 - GlobalExceptionHandler를 통해 런타임 예외를 중앙에서 관리
 - 커스텀 예외(BaseException) 도입
 - 클라이언트에게 명확한 에러 코드와 메시지 전달
 - 예: EMAIL_NOT_VERIFIED, DUPLICATE_USER 등
 - 효과: 프론트엔드에서의 디버깅 용이성 및 사용자 경험(UX) 향상

```
import org.springframework.http.HttpStatus;  
import org.springframework.web.bind.annotation.ResponseStatus;  
  
@ResponseStatus(HttpStatus.BAD_REQUEST)  
public class BadRequestException extends RuntimeException {  
    public BadRequestException(String message) {  
        super(message);  
    }  
}
```

인증	
DELETE	/api/v1/auth/logout 로그인 API
GET	/api/v1/auth/check-email 이메일 중복 체크 API
POST	/api/v1/auth/verify-code 비밀번호 인증
POST	/api/v1/auth/signup 회원가입 API
POST	/api/v1/auth/send-mail/email 이메일 발송
POST	/api/v1/auth/refresh 토큰 재발급 API
POST	/api/v1/auth/login 로그인 API

```

// GET /api/status/:id
API.GET('/api/status/:id', async (req, res) => {
  const { id } = req.params;
  const status = await Status.findById(id);
  if (!status) return res.status(404).json({ message: 'Status not found' });
  res.json(status);
});

// GET /api/status/:id/updates
API.GET('/api/status/:id/updates', async (req, res) => {
  const { id } = req.params;
  const status = await Status.findById(id);
  if (!status) return res.status(404).json({ message: 'Status not found' });
  const updates = await Update.find({ status: id }).sort({ createdAt: -1 });
  res.json(updates);
});

// POST /api/status/:id/updates
API.POST('/api/status/:id/updates', async (req, res) => {
  const { id } = req.params;
  const { text } = req.body;
  if (!text) return res.status(400).json({ message: 'Text is required' });
  const update = await Update.create({ status: id, text });
  res.json(update);
});

// DELETE /api/status/:id
API.DELETE('/api/status/:id', async (req, res) => {
  const { id } = req.params;
  const status = await Status.findById(id);
  if (!status) return res.status(404).json({ message: 'Status not found' });
  await status.delete();
  res.json({ message: 'Status deleted' });
});

```

Backend 개발방법

핵심기능

3. 데이터 전송 및 저장

- 1) 대시보드 데이터 전송 : 백엔드에 저장된 위험 분석 결과와 통계 데이터를 프론트엔드(대시보드) 화면에 시각화할 수 있는 형태로 전달
- 2) 고객 데이터 전송: 고객의 개인정보와 위험 요인 등의 데이터를 프론트엔드로 전달
- 3) AI 분석 데이터 저장: AI 모델에서 나온 결과 값을 DB에 저장

4. API 문서화

Swagger UI (OpenAPI)

- 자동화된 문서: 코드 변경 시 API 문서 자동 업데이트
- 테스트 용이성: 프론트엔드 개발자가 백엔드 로직 없이도 API 테스트 가능
- 명세 공유: Request/Response DTO 구조를 명확히 시각화

dashboard-controller

GET /api/v1/dashboard/statistics

GET /api/v1/dashboard/customers

GET /api/v1/dashboard/customer/{customerId}

데이터 저장 데이터 저장 API

POST /api/v1/dataset/save

API 문서화 (Swagger UI)

API	/api/v1/dashboard/statistics	GET
API	/api/v1/dashboard/customers	GET
API	/api/v1/dashboard/customer/{customerId}	GET
API	/api/v1/dataset/save	POST

리스크

API	/api/v1/risk/analyze	POST
-----	----------------------	------

대용

API	/api/v1/data/import	POST
-----	---------------------	------

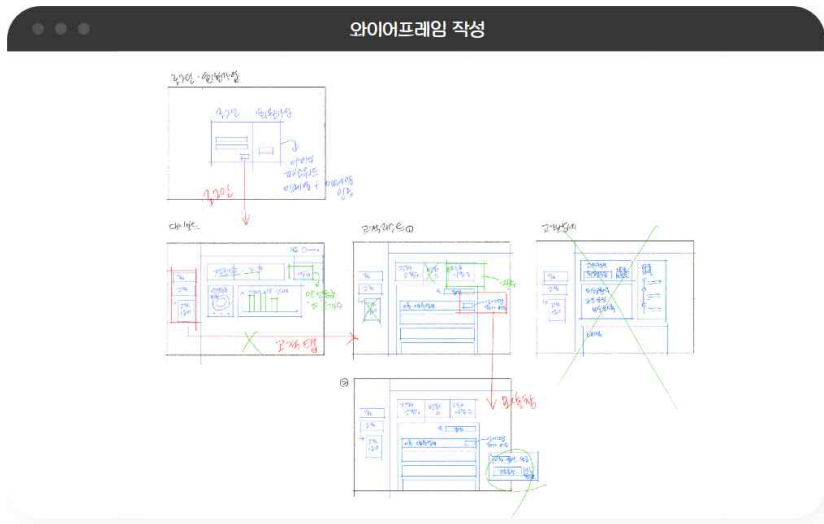
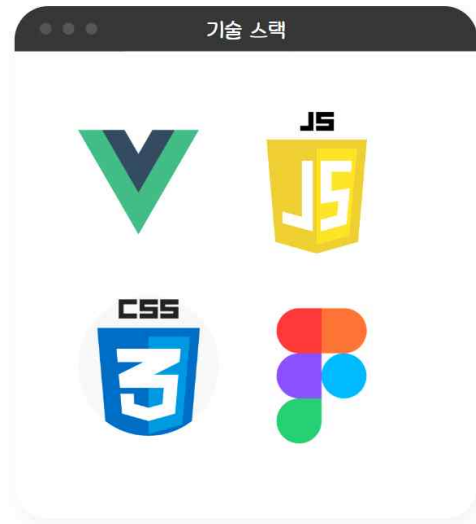
데이터 저장

API	/api/v1/dataset/save	POST
-----	----------------------	------

dashboard-controller

API	/api/v1/dashboard/statistics	GET
API	/api/v1/dashboard/customers	GET
API	/api/v1/dashboard/customer/{customerId}	GET

Frontend 개발방법



목표 정의 -> 요구사항 분석 -> 와이어프레임 작성 -> 디자인/CSS 작업 -> 웹사이트 구현 -> API 연동/백엔드 연결



로그인/회원가입 페이지

 Risk Finder

Log In

아이디

비밀번호

1 로그인하기

로그인

db에 저장된 회원정보로
로그인합니다.

아직 회원이 아니신가요?

회원가입

2

회원가입

가입된 회원이 아니라면
회원가입 페이지로 이동합니다.



로그인/회원가입 페이지

Sign Up

아이디

비밀번호 입력

비밀번호 재입력

이메일 입력

인증번호 입력

인증번호 전송

인증 완료

가입하기

1

이메일 인증

이메일 인증 후
회원가입 가능합니다.

회원인가요?

로그인

2

회원가입하기

정보를 입력하고
버튼을 누르면
로그인페이지로 이동합니다.



대시보드



Risk Finder

Dashboard

고객의 점수와 등급을 한눈에 확인하세요.

Admin ▾

Dashboard

Customer

6 고객 페이지 이동

버튼을 누르면 고객 정보가 있는 페이지로 이동합니다.

4 안전등급 비율 시각화

안전등급별 고객 비율을 도넛차트로 시각화합니다.

0.5012

외부 소스1 평균 점수

1.00 +1.00% 이변주

0.5180

외부 소스2 평균 점수

2.00 -0.90% 이변주

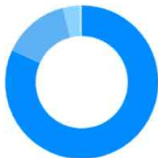
0.4112

외부 소스3 평균 점수

2.00 -0.90% 이변주

안전등급 비율

2025년 11월 기준



- A등급 81.9%
- B등급 14.2%
- C등급 3.5%
- D등급 0.5%
- E등급 0.0%

0.01%

안전등급 E 고객 비율

3.00 +1.00% 이변주

6.50%

전체 고객 부도확률

3.00 +1.00% 이변주

전체 고객 부도확률

최근 6개월 기준



1 외부 기관 평균 점수 조회

외부 기관에서 평가한 금융 평가 점수의 평균을 조회합니다.

2 E등급 고객 비율 조회

안전등급이 E등급인 고객 비율을 조회합니다.

3 전체 고객 부도확률

전체 고객 부도확률을 조회합니다.

5 부도확률 history 시각화

6개월간 모니터링된 부도확률 history를 시각화합니다.



고객 페이지



Dashboard

Customer

4 고객 정보

고객 아이디, 이름, 안전등급, 부도확률을 보여줍니다.

아이디	이름	안전등급	부도확률
456250	서명자	B (양호)	11.37%
456224	오지후	A (안전)	3.26%
456223	김정웅	A (안전)	1.79%
456222	윤서현	A (안전)	8.46%
456221	홍성민	A (안전)	3.51%
456202	박정식	A (안전)	6.47%
456189	윤민재	A (안전)	6.81%

고객 아이디로 고객 조회

안전 - 등급

1 등급별 고객 조회

등급별로 고객을 필터링하여 조회합니다.

2 고객 아이디 검색

고객 아이디를 통해 고객을 찾습니다.

3 고객 상세정보 조회

버튼을 누르면 고객 상세정보 모달 창이 생성됩니다.

Contact



고객 상세 정보

456224 오지후 A (안전) 3.26% 적용하기

456223

456222

456221

456202

456189

456170

456169 김영호 A (안전) 4.37%

456168 이승현 A (안전) 4.26%

1 of 4875 pages (Total: 48744)

Contact

1 2 3 4 5 >

456223

안전등급 부도확률
A (안전) 1.79%

안내 매일 발송

이름 김정웅
전화번호 011-869-4291
이메일 gimhaeun@example.net

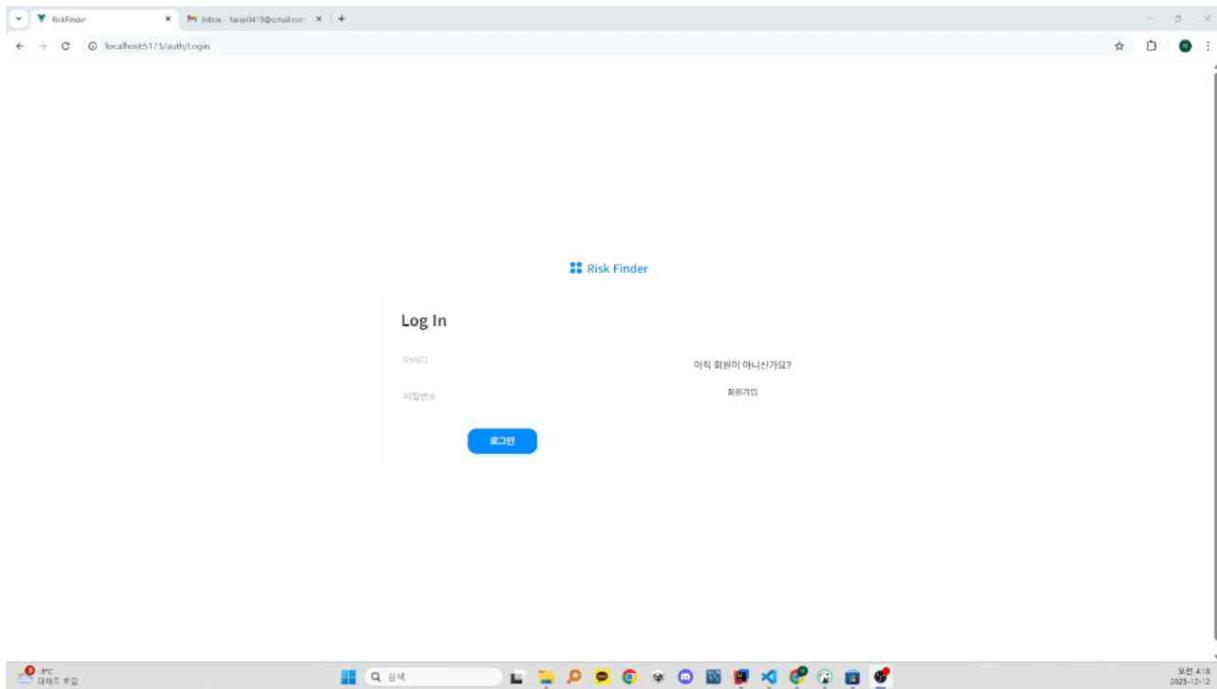
주요요인 (Top 3)

- ANNUITY_TO_CREDIT_RATIO
- EXT_SOURCE_MEAN
- bureau_DAYS_CREDIT_max

1 고객 상세정보 모달

고객에 대한 모든 정보가 들어 있습니다. 여기서 안내 메일을 발송할 수 있습니다.

시연 영상



차별점

기존 시스템

대출 실행 시점 심사 집중
결과만 통보하는 Black-box



대출 실행 이후 실시간 모니터링 특화
위험 원인을 밝히는 White-box
필요할 때만 쓰는 serverless로 비용 최적화

기대 효과

1. 재무적 건전성 및 비용 최적화

- 부실채권(NPL) 사전 차단
- Infra 비용 절감

2. 업무 프로세스 자동화

- Human Error 방지
- 생산성 극대화

3. 신뢰 기반의 리스크 관리

- 설명 가능한 AI
- 선제적 케어

chapter 4

협업 과정

팀 소개

협업 과정

신서현

컴퓨터인공지능학부

인프라 설계 / DevOps / DB

PPT 제작

박준영

컴퓨터공학부

백엔드 개발 / 프론트엔드 주요 페이지 개발

최종 프로젝트 발표

박민호

컴퓨터공학부

AI 모델 개발

임다미

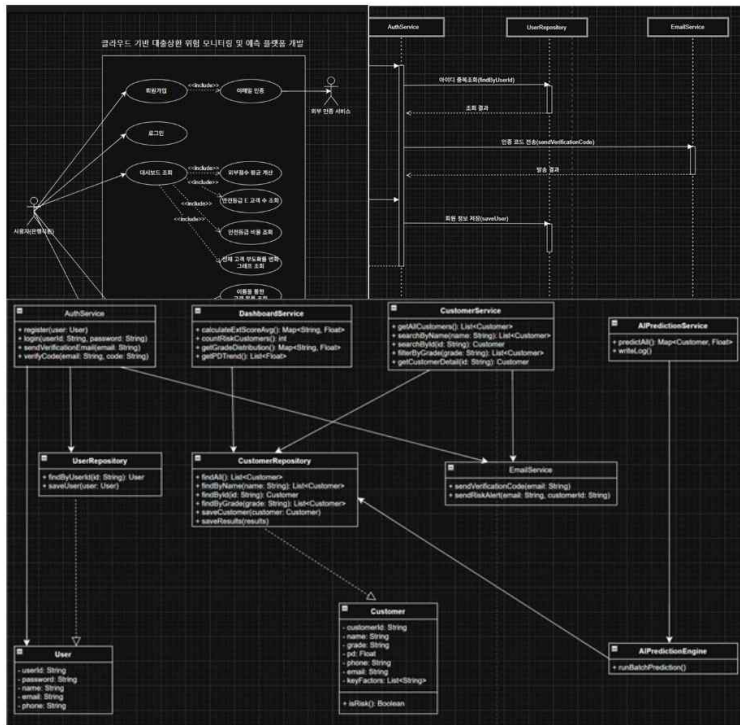
컴퓨터공학부

웹 디자인 / 프론트엔드 개발

1

요구사항 분석 및 설계

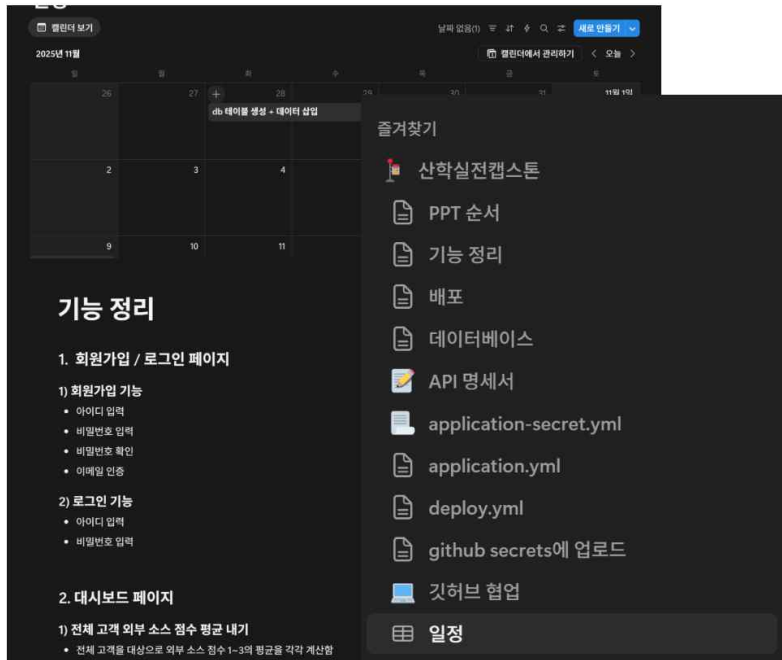
- 요구사항 정의: Use Case Diagram & Scenario
- 시스템 설계: Class Diagram & Sequence Diagram
- 산출물 정리: 분석 내용을 총망라한 요구사항 명세서 완성



2

협업 & 일정 관리 (Notion, github)

- 일정 시각화: 캘린더를 활용한 프로젝트 로드맵 및 진행도 실시간 공유
- 개발 자산 통합: 서비스 기능 명세, API 문서 등 핵심 파일 아카이빙
- 환경 정보 공유: DB 연결 정보, 배포 주소 등 개발 리소스 일원화
- 코드 관리 : 개발 코드를 github organization을 통해 관리



감사합니다